

Operating Systems

2520030227

N.Prudvi Raj

SEC-5

WEEK-4

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Code

```
prudvi@LAPTOP-TEK0376B: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t p1, p2;  
    p1 = fork();  
    if (p1 == 0)  
    {  
        printf("Child 1 PID: %d\n", getpid());  
        sleep(2);  
        printf("Child 1 completed\n");  
        return 0;  
    }  
  
    p2 = fork();  
    if (p2 == 0)  
    {  
        printf("Child 2 PID: %d\n", getpid());  
        sleep(4);  
        printf("Child 2 completed\n");  
        return 0;  
    }  
  
    printf("Parent PID: %d\n", getpid());  
    wait(NULL);  
    printf("One child completed using wait()\n");  
    waitpid(p2, NULL, 0);  
    printf("Child 2 completed using waitpid()\n");  
    return 0;  
}
```

output

```
prudvi@LAPTOP-TEK0376B:~$ nano child.c  
prudvi@LAPTOP-TEK0376B:~$ gcc child.c  
prudvi@LAPTOP-TEK0376B:~$ ./a.out  
Child 1 PID: 674  
Parent PID: 673  
Child 2 PID: 675  
Child 1 completed  
One child completed using wait()  
Child 2 completed  
Child 2 completed using waitpid()
```